

جامعة الأمير سلطان
PRINCE SULTAN
UNIVERSITY

IS231 Project

PSU Campus Preorder System

Instructor: Dr. Amerah Alghanim

Done by:

Latifa Albabtain -

Daad Aljaber -

Leena Alarifi -

Sitah Alsemmari -

Phase 1:

1- Domain Discovery:

Stakeholders:

- Customers (Students, Faculty, Staff): end-users who will browse menus, place orders, and pick up food.
- Restaurant staff: responsible for receiving orders, preparing meals, and marking them as ready for pickup.
- IT Department/ System Admin: ensure the system is running smoothly, manage updates, security, and integration with university systems.
- University Management: interested in reducing congestion in the Food Hub and improving student satisfaction.
- Payment Gateway Provider: external stakeholders ensuring secure financial transactions.

Research:

We observed the daily crowding at the Food Hub, especially during break time, where long queues cause noticeable delays. We also spoke with several students who mentioned that the waiting time often makes them late for their classes, while others admitted they sometimes skip meals to avoid the hassle of waiting. These insights highlighted the need for a streamlined preorder system that saves time for students and makes operations easier for the restaurant.

2- Vision Document:

Purpose: The PSU Campus Preorder System aims to enhance the dining experience at Prince Sultan University's Food Hub by reducing wait times, traffic, and tension during busy times. The system offers a web-based platform that allows consumers to pay online, see menus, and place food preorders. Because orders will be prepared ahead of time, consumers will not have to wait in long queues to pick up their meals. This guarantees an improved smooth service flow and minimized errors.

Target users: The system is designed for everyone within the Prince Sultan University campus community, including:

• Students – who need quick meal access between classes.

• Faculty and Staff – who want efficient ordering without interrupting their schedules.

• Restaurant Staff – who will benefit from better order management, fewer mistakes, and reduced crowd pressure.

Goals:

1. Enhance User Experience
   - Provide a fast, convenient, and reliable way to order and pay for food.
   - Reduce stress and lateness caused by long queues.
2. Optimize Restaurant Operations
   - Enable restaurants to handle orders more systematically with scheduled pickups.
   - Reduce errors caused by rushed in-person ordering.
3. Improve Campus Efficiency

- Minimize congestion in the Food Hub during peak hours.
- Support smoother traffic flow and better time management for the PSU community.

Success Criteria

The project will be considered successful if:

- Reduced Wait Times: Average physical wait time for food pickup decreases by at least 50% compared to pre-implementation measurements.
- Adoption Rate: At least 70% of students, faculty, and staff place 2 or more pre-orders through the system within the first semester.
- Restaurant Efficiency: Food vendors report a reduction in order errors (e.g., wrong items, missing items) by at least 30% within the first 3 months of implementation.
- User Satisfaction: At least 80% of users who try the system place 3 or more repeat orders, indicating satisfaction with the experience.
- Sustained Usage: After 6 months, the pre-order system accounts for at least 60% of all on-campus food transactions, showing that it has become the preferred ordering method.

3- Supplementary Specification:

The Supplementary Specification document describes the non-functional requirements, constraints, and additional specification for the PSU campus preorder system that are not addressed in the use cases or system vision document. The document assures the system meets Prince Sultan University's technological and operational requirements.

Functional requirements

1. Authentication and User Account:

- Allow users (staff, faculty, and students) to sign in using their PSU login credentials (single log-on).
- Provide a secure password recovery mechanism with an institution validated email address.
- Access based on role is supported for users, employees, and administrators.

2. Browsing and ordering:
    - View current menu items, pricing, and availability.
    - Allow clients to customize their orders by adding or removing things.
    - Make room for urgent or scheduled order pickup.
3. Check-Out and pay:
    - Make safe online payments with an integrated gateway.
    - Provide detailed billing and tax details before receiving confirmation.
    - Orders and payments can be confirmed via email or app notifications.
4. Order management:
    - Allow restaurant staff to accept, process, and label orders as “ready for pickup”.
    - Allow orders to be cancelled or modified prior to the preparation threshold.
5. Analysis and reporting:
    - Provide administrators with information on peak hours, order frequency, and user comments.
    - Keep track of all order related activities for future audits.

Non-Functional requirements

1. **Usability**
    - Provide a mobile friendly user interface that is optimized for IOS and Android.

2. **Reliability**
    - Failed transactions should automatically initiate user notifications.
    - The system must handle orders with 99.5% reliability, guaranteeing that no transactions or data are lost during daily operation.
    - It is important for order processing components to have fault tolerance designs.
    - Incomplete or unsuccessful order logs must be kept for analysis.
3. **Performance**
    - The system must be able to support at least 500 concurrent users without any latency.
    - Under normal network conditions, page loading time should be no more than 2 seconds.
    - Order processing and confirmation must be done in less than three seconds.
4. **Security**

    Authentication:

    - Make all users use university Single SignOn (SSO).
    - Every semester, administrators, staff, and students should update their passwords more often.

    Data Encryption:

    - Encrypt user data that is saved (payment details, order history).

    Access Control:

    - For system users, put Role Based Access Control into practice.

- All administrative acts must be documented in audit logs.

5. **Scalability**
    - Dynamically scale infrastructure to support up the baseline peak load.
    - Allow for the onboarding of several campus sites in the future without requiring rearchitecture.
    - Continue to perform consistently even if the number of orders or users doubles.
6. **Maintability**
    - Isolate functional components using a modular architecture.
    - Neat and well documented codebase using version control.
    - Include administrative dashboards for system monitoring and log review.
7. **Compatibility**
    - Compatible with all modern browsers (Safari ,Chrome).
    - Integrate smoothly into PSU systems (student database).
    - Payment gateway must support local and international card processors.
8. **Compliance**
    - Ensure compliance with the NCA (National Cybersecurity Authority) Cybersecurity Framework and PSU IT governance policies.
    - Follow data handling practices consistent with GDPR-like privacy standards for student and staff data.
    - Payment integration must comply with PCI DSS for secure transactions.

4- Glossary:

Preorder System: A platform that lets users place food orders in advance and pick them up without waiting in long lines.

• **Stakeholders:** All parties involved in or benefiting from the system, including students, faculty, staff, restaurants, and university management.

• **End-Users:** The main users of the system, primarily students, faculty, and staff.

• **Single Sign-On (SSO):** A login method that allows users to access the system using their PSU credentials.

• **Payment Gateway:** An external service that securely processes online payments.

• **Order Management:** The process of receiving, preparing, and marking food orders as ready for pickup by restaurant staff.

• **Role-Based Access Control (RBAC):** A security method that assigns system access based on user roles, such as student, staff, or administrator.

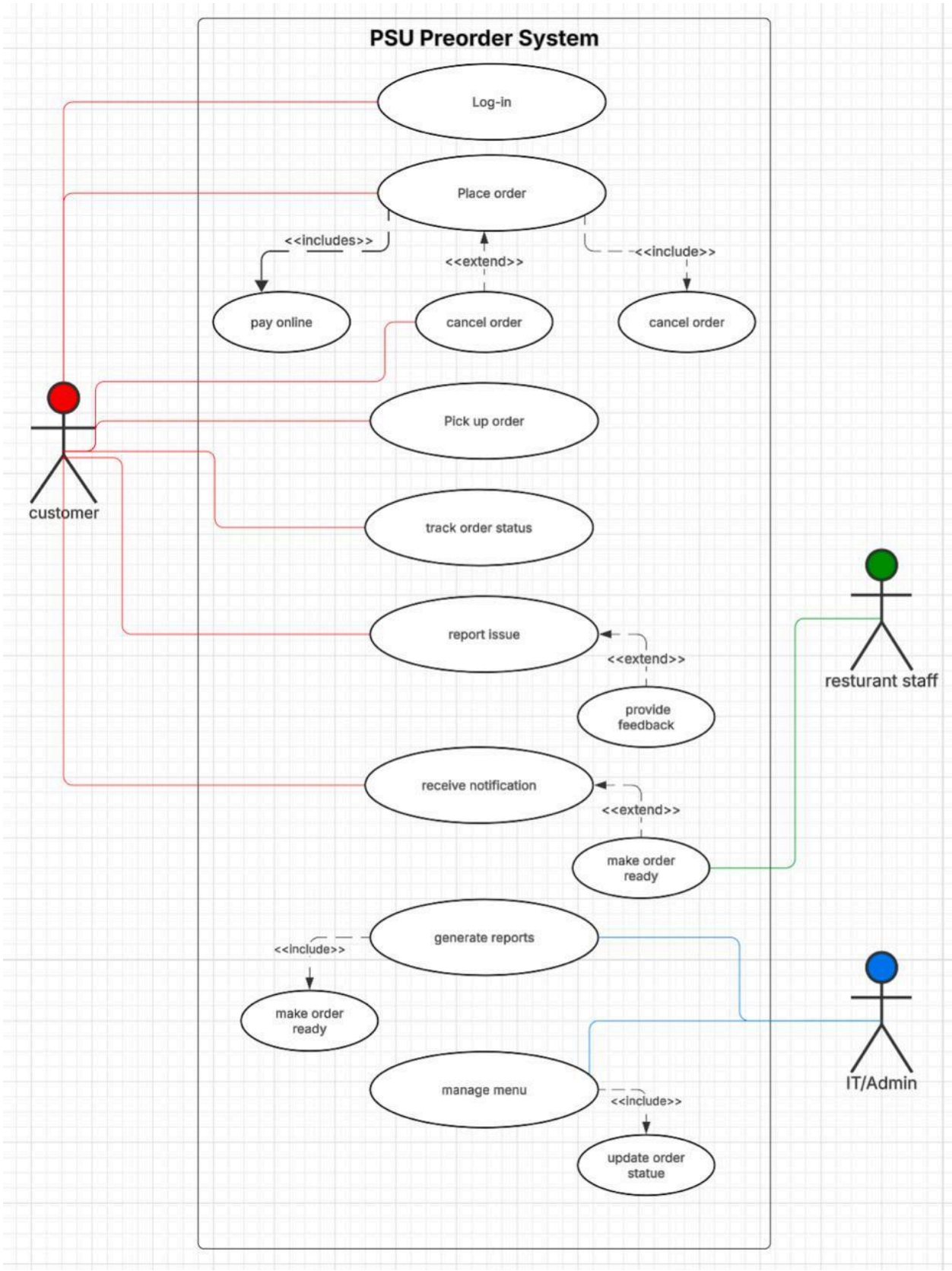
• **Non-Functional Requirements (NFRs):** System qualities that are not related to specific features, including security, reliability, and performance.

• **Scalability:** The system's ability to manage growth, like more users or additional campus sites, without major redesign.

• **Audit Logs:** Records of administrative and system activities used for monitoring and security reviews.

5- Diagrams:

- **Use-Case Diagram:**


PSU Preorder System
Log-in
Place order
<<includes>>
<<extend>>
<<include>>
pay online
cancel order
cancel order
customer
Pick up order
track order status
report issue
<<extend>>
provide feedback
resturant staff
receive notification
<<extend>>
make order ready
generate reports
<<include>>
make order ready
IT/Admin
manage menu
<<include>>
update order statue

Phase 2

1. User Story:

1. As a **student**, I want to be able to log in using my PSU account to access the preorder system, so that I can browse menus, select my meal, and preorder it easily without waiting in line.
2. As a **student**, I want to be able to pay online through a secure payment gateway, so that I can confirm my order easily and receive a notification when it's ready.
3. As a **system administrator**, I want to be able to monitor order activities, system performance, manage menu updates, and generate reports, so that I can ensure smooth operations, track restaurant performance, and quickly fix any issues that occur.
4. As a **restaurant staff member**, I want to receive and update customer orders in real time, so that I can prepare meals on time and mark them as "ready for pickup" so that customers are notified immediately.
5. As a **faculty member**, I want to be able to preorder my meal online, so that I can pick it up quickly without affecting my class or meeting schedule.

2. Acceptance criteria (Given/ When/ Then)

User Story 1.

• **Given** the student is a PSU member with valid credentials, **When** they enter their PSU email and password, **Then** the system should authenticate them via SSO and allow access to the preorder dashboard.

• **Given** the student provides invalid credentials: **When** clicking the log-in button,**Then** an error message should pop up without access being given.

• **Given** the student forgets their password, **When** they click "Forgot Password," **Then** a recovery e-mail should be sent to their PSU address.

User Story 2.

• **Given** the student selects items and proceeds to checkout; **When** choosing an online payment option and confirming, **Then** the payment gateway should process the transaction securely and show a confirmation message.

• **Given** the payment is successful, **When** the transaction is complete, **Then** system should generate an order ID and send a confirmation of the order via email/app notification.

• **Given** the payment fails, **When** the transaction is declined or interrupted, **Then** the system should cancel the order and notify the user.

User Story 3

• **Given** the administrator is logged in, **When** they open the admin dashboard, **Then** they should see key metrics, such as order volume, restaurant performance, and uptime.

• **Given** the admin chooses “Reports,” **When** they generate a report for a certain period, **Then** a downloadable report should be generated that shows summarized data.

• **Given** menu changes are needed, **When** the admin makes edits or deletions of items, **Then** the menu should be refreshed instantaneously to the user.

User Story 4

• **Given** a student places an order, **When** the restaurant dashboard refreshes, **Then** the new order should appear instantly with all order details.

• **Given** an order is being prepared, **When** the staff marks it as "Ready for Pickup," **Then** the customer should immediately receive a notification.

• **Given** the status of an order changed, **when** the staff updates or cancels it, **then** the system should log the change and notify the customer.

User Story 5

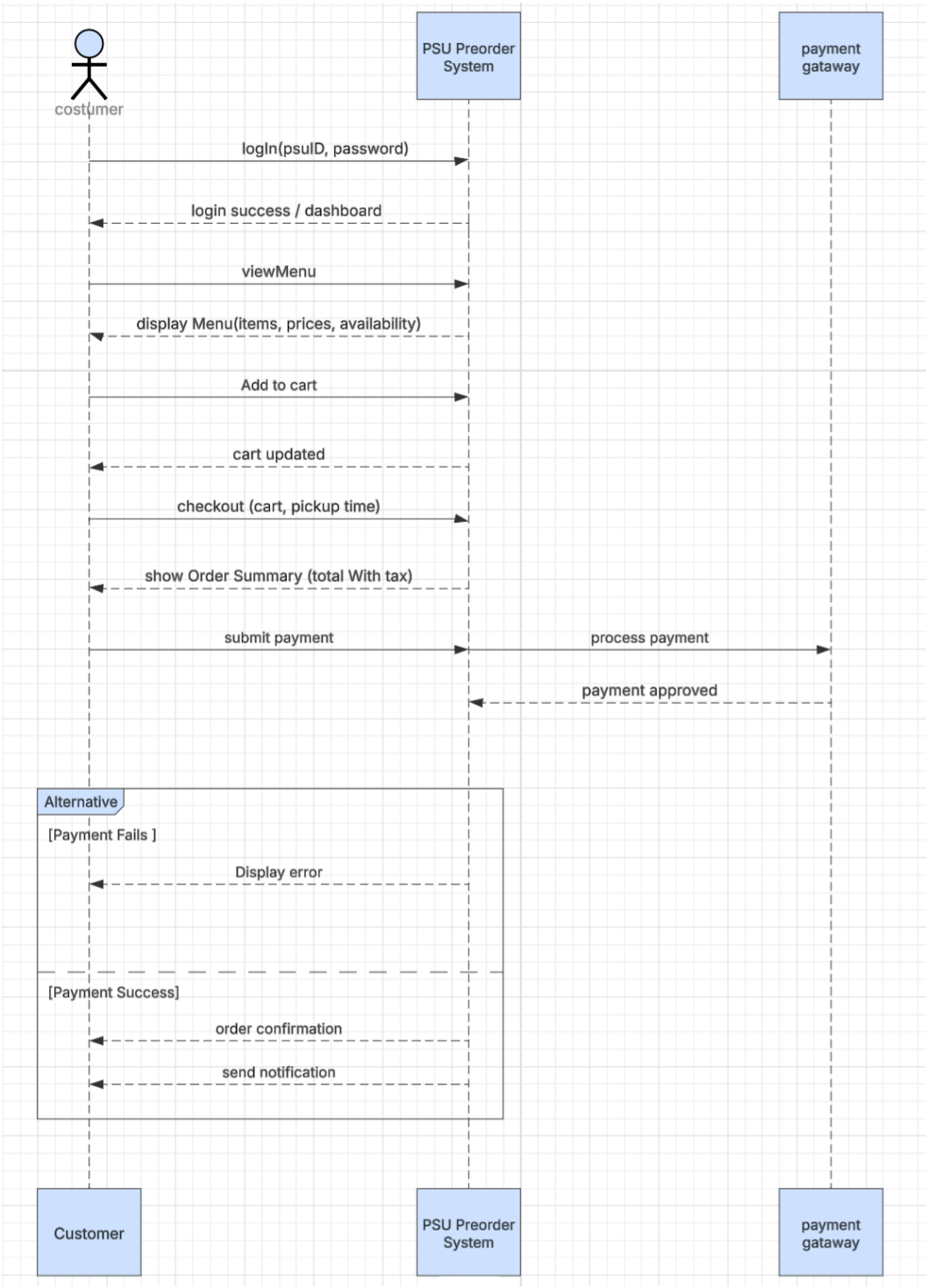
• **Given** the faculty member is logged in, **When** they select and preorder a meal, **Then** the system should confirm the order and show the scheduled pickup time.

• **Given** that the order might need to be canceled or revised by the faculty member, **When** the change is made before preparation starts, **Then** it should automatically update in the restaurant dashboard.

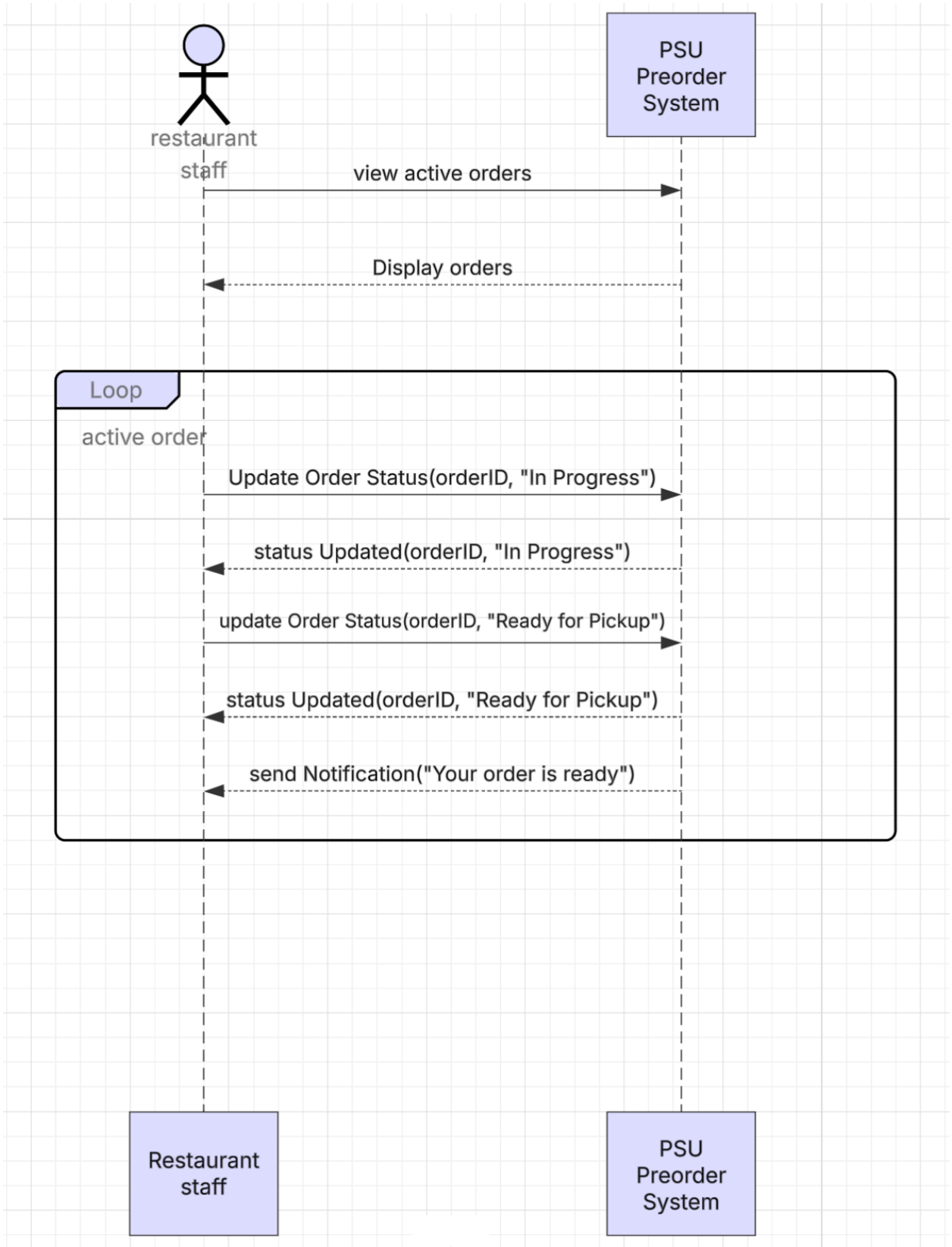
• **Given** the order is ready, **When** the faculty member checks the system, **Then** a "Ready for Pickup" notification should be displayed.

4. Diagrams (UML tool):

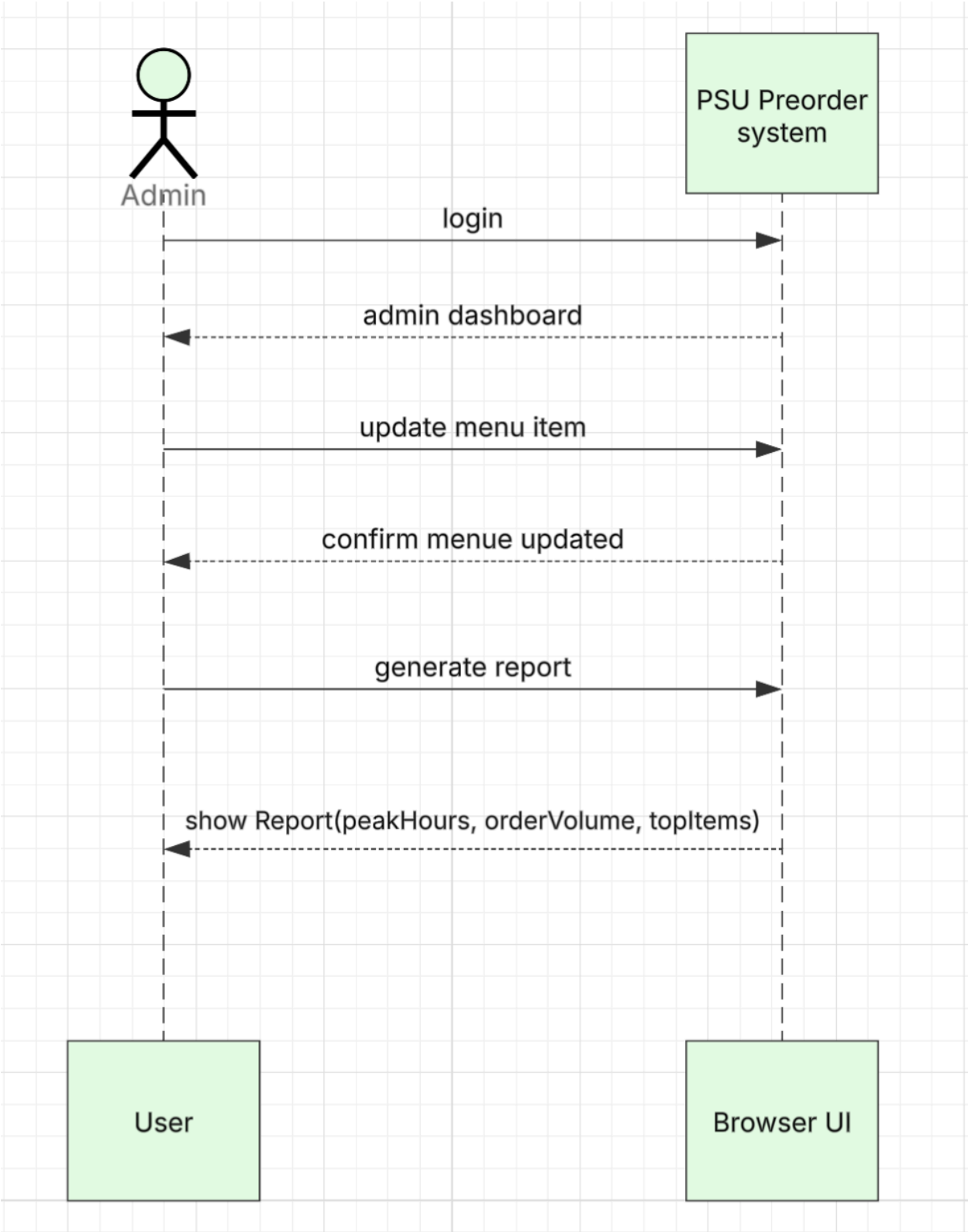
SSD1 (Place and pay for order):



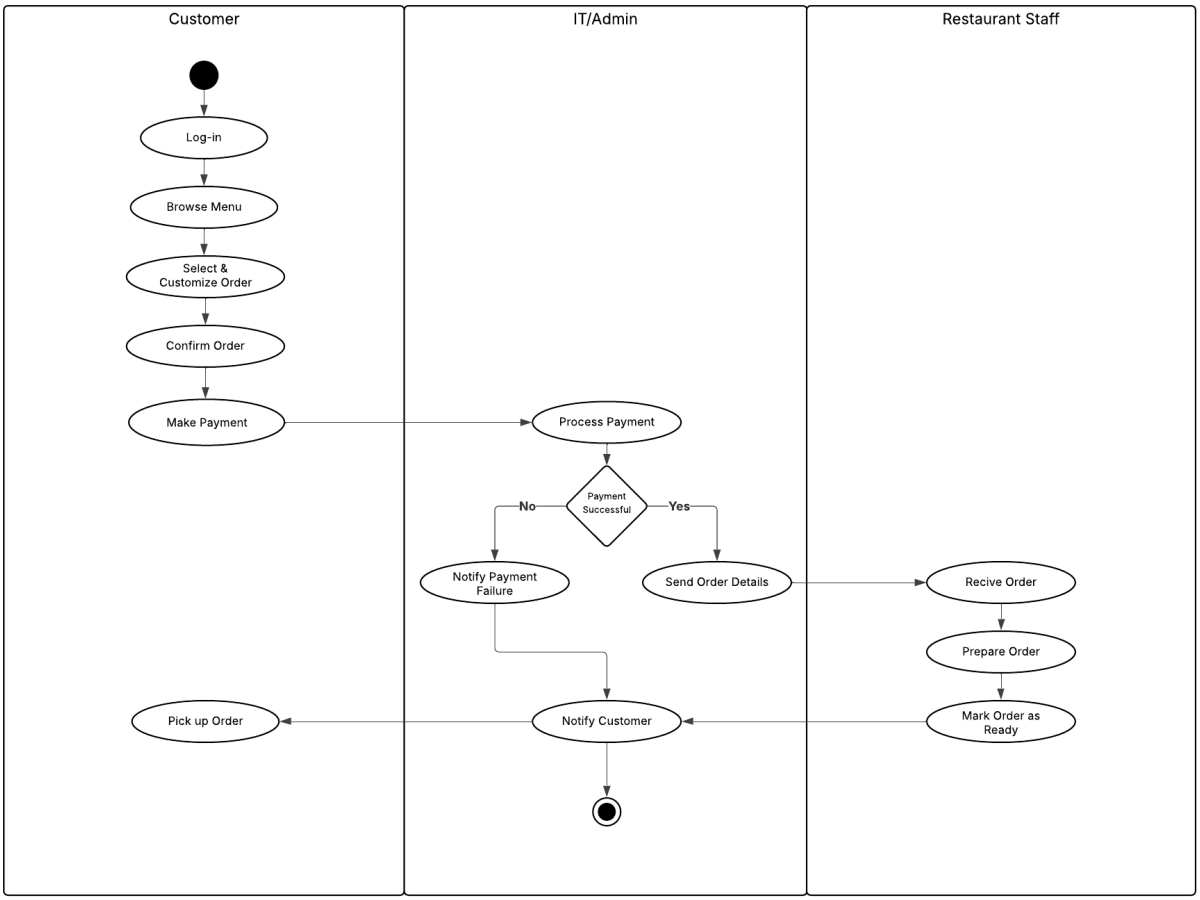
SSD2 (make order ready):



SSD3 (The admin view reports and update menu):


Admin
PSU Preorder system
login
admin dashboard
update menu item
confirm menue updated
generate report
show Report(peakHours, orderVolume, topItems)
User
Browser UI

- **Activity Diagram(s) — for complex workflows.**



CRC cards:

User	
• Log in / out using PSU • Manage profile and view order history • Browse menu and place orders	Order NotificationService

Order	
• Create, modify or cancel preorders • Manage order status (placed, in progress, ready) • Calculate total and confirm payment • Notify user upon status change	User

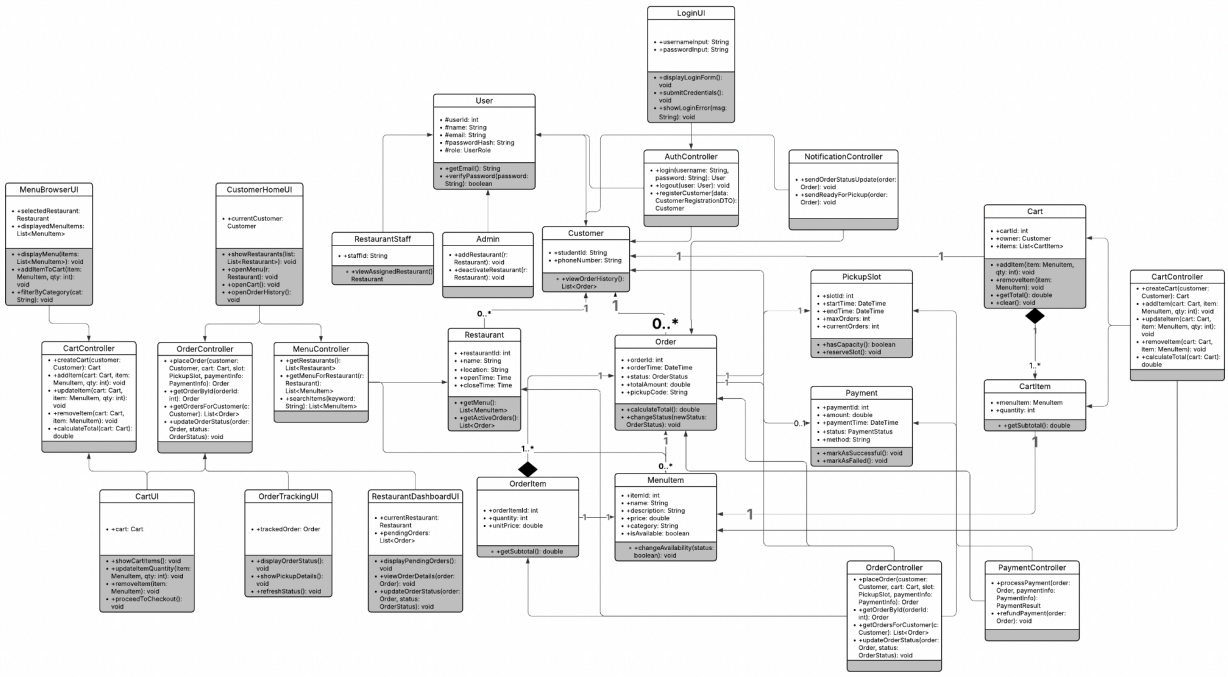
Menu	
• Display available items and prices • Allow customization (add/remove items)	Order User

Admin	
• Manage menu content • Monitor system activites • Generate reports	Menu Order Report

ResturantStaff	
• View incoming orders in real time • Mark them as ready for pick up • Update order status	Order User

NotificationService	
• Sending notification (email, application)	Admin

DCD:


LoginUI
+usernameInput: String
+passwordInput: String
+displayLoginForm(): void
+submitCredentials(): void
+showLoginError(msg: String): void
User
#userId: int
#name: String
#email: String
#passwordHash: String
#role: UserRole
+getEmail(): String
+verifyPassword(password: String): boolean
AuthController
+loginUsername: String, password: String): User
+logout(user: User): void
+registerCustomer(data: CustomerRegistrationDTO): Customer
NotificationController
+sendOrderStatusUpdate(order: Order): void
+sendReadyForPickup(order: Order): void
MenuBrowserUI
+selectedRestaurant: Restaurant
+displayedMenuItems: List<MenuItem>
+displayMenuItems: List<MenuItem>): void
+addItemToCart(item: MenuItem, qty: int): void
+filterByCategory(cat: String): void
CustomerHomeUI
+currentCustomer: Customer
+showRestaurants(list: List<Restaurant>): void
+openMenu(r: Restaurant): void
+openCart(): void
+openOrderHistory(): void
RestaurantStaff
+staffId: String
+viewAssignedRestaurant(): Restaurant
Admin
+addRestaurant(r: Restaurant): void
+deactivateRestaurant(r: Restaurant): void
Customer
+studentId: String
+phoneNumber: String
+viewOrderHistory(): List<Order>
Cart
+cartId: int
+owner: Customer
+items: List<CartItem>
+addItem(item: MenuItem, qty: int): void
+removeItem(item: MenuItem): void
+getTotal(): double
+clear(): void
CartController
+createCart(customer: Customer): Cart
+addItem(cart: Cart, item: MenuItem, qty: int): void
+updateItem(cart: Cart, item: MenuItem, qty: int): void
+removeItem(cart: Cart, item: MenuItem): void
+calculateTotal(cart: Cart): double
OrderController
+placeOrder(customer: Customer, cart: Cart, slot: PickupSlot, paymentInfo: PaymentInfo): Order
+getOrderById(orderId: int): Order
+getOrdersForCustomer(c: Customer): List<Order>
+updateOrderStatus(order: Order, status: OrderStatus): void
MenuController
+getRestaurants(): List<Restaurant>
+getMenuForRestaurant(r: Restaurant): List<MenuItem>
+searchItems(keyword: String): List<MenuItem>
Restaurant
+restaurantId: int
+name: String
+location: String
+openTime: Time
+closeTime: Time
+getMenu(): List<MenuItem>
+getActiveOrders(): List<Order>
Order
+orderId: int
+orderTime: DateTime
+status: OrderStatus
+totalAmount: double
+pickupCode: String
+calculateTotal(): double
+changeStatus(newStatus: OrderStatus): void
PickupSlot
+slotId: int
+startTime: DateTime
+endTime: DateTime
+maxOrders: int
+currentOrders: int
+hasCapacity(): boolean
+reserveSlot(): void
Payment
+paymentId: int
+amount: double
+paymentTime: DateTime
+status: PaymentStatus
+method: String
+markAsSuccessful(): void
+markAsFailed(): void
CartItem
+menuItem: MenuItem
+quantity: int
+getSubtotal(): double
CartController
+createCart(customer: Customer): Cart
+addItem(cart: Cart, item: MenuItem, qty: int): void
+updateItem(cart: Cart, item: MenuItem, qty: int): void
+removeItem(cart: Cart, item: MenuItem): void
+calculateTotal(cart: Cart): double
CartUI
+cart: Cart
+showCartItems(): void
+updateItemQuantity(item: MenuItem, qty: int): void
+removeItem(item: MenuItem): void
+proceedToCheckout(): void
OrderTrackingUI
+trackedOrder: Order
+displayOrderStatus(): void
+showPickupDetails(): void
+refreshStatus(): void
RestaurantDashboardUI
+currentRestaurant: Restaurant
+pendingOrders: List<Order>
+displayPendingOrders(): void
+viewOrderDetails(order: Order): void
+updateOrderStatus(order: Order, status: OrderStatus): void
OrderItem
+orderItemId: int
+quantity: int
+unitPrice: double
+getSubtotal(): double
MenuItem
+itemId: int
+name: String
+description: String
+price: double
+category: String
+isAvailable: boolean
+changeAvailability(status: boolean): void
OrderController
+placeOrder(customer: Customer, cart: Cart, slot: PickupSlot, paymentInfo: PaymentInfo): Order
+getOrderById(orderId: int): Order
+getOrdersForCustomer(c: Customer): List<Order>
+updateOrderStatus(order: Order, status: OrderStatus): void
PaymentController
+processPayment(order: Order, paymentInfo: PaymentInfo): PaymentResult
+refundPayment(order: Order): void
1
0..*
1..*